

TERM PROJECT 2

CODE:

```
1 from LinkedBinaryTree import LinkedBinaryTree as Tree
2
3 tree = Tree()
4 print("Vertex: r a b c d e f g h")
5 print("L.C    a b - e - - - - ")
6 print("R.C    - c d f - g - h - ")
7 tree._add_root('r')
8 tree._add_left(tree.root(), e: 'a')
9 tree._add_left(tree.left(tree.root()), e: 'b')
10 tree._add_right(tree.left(tree.root()), e: 'c')
11 tree._add_right(tree.left(tree.left(tree.root())), e: 'd')
12 tree._add_left(tree.right(tree.left(tree.root())), e: 'e')
13 tree._add_right(tree.right(tree.left(tree.root())), e: 'f')
14 tree._add_right(tree.left(tree.right(tree.left(tree.root()))), e: 'g')
15 tree._add_right(tree.right(tree.left(tree.right(tree.left(tree.root())))), e: 'h')
16
17 #use the preorder traversal to print the tree
18 print("Preorder traversal: ", end = " ")
19 for i in tree.positions():
20     print(i.element(), end = " ")
21 print()
22
23 #use the post order traversal to print the tree
24 print("Postorder traversal: ", end = " ")
25 for i in tree.postorder():
26     print(i.element(), end = " ")
27 print()
28 print()
29
30 tree = Tree()
31 print("Vertex: r a b c d e f g")
32 print("L.C    a c - - - - - ")
33 print("R.C    b d e - - f g - ")
34 tree._add_root('r')
35 tree._add_left(tree.root(), e: 'a')
36 tree._add_right(tree.root(), e: 'b')
37 tree._add_left(tree.left(tree.root()), e: 'c')
38 tree._add_right(tree.left(tree.root()), e: 'd')
39 tree._add_right(tree.right(tree.root()), e: 'e')
```

```
41 tree._add_right(tree.right(tree.right(tree.root())), e: 'f')
42 tree._add_right(tree.right(tree.right(tree.right(tree.root()))), e: 'g')
43
44 #use the preorder traversal to print the tree
45 print("Preorder traversal: ", end = " ")
46 for i in tree.positions():
47     print(i.element(), end = " ")
48 print()
49
50 #use the post order traversal to print the tree
51 print("Postorder traversal: ", end = " ")
52 for i in tree.postorder():
53     print(i.element(), end = " ")
54 print()
55 print()
56
57 tree = Tree()
58 print("Vertex: r a b c d e f")
59 print("L.C    a - d f - - - ")
60 print("R.C    b c e - - - - ")
61 tree._add_root('r')
62 tree._add_left(tree.root(), e: 'a')
63 tree._add_right(tree.root(), e: 'b')
64 tree._add_left(tree.left(tree.root()), e: 'c')
65 tree._add_left(tree.right(tree.root()), e: 'd')
66 tree._add_right(tree.right(tree.root()), e: 'e')
67 tree._add_left(tree.left(tree.left(tree.root()))), e: 'f')
68
69
70 #use the preorder traversal to print the tree
71 print("Preorder traversal: ", end = " ")
72 for i in tree.positions():
73     print(i.element(), end = " ")
74 print()
75
76 #use the post order traversal to print the tree
77 print("Postorder traversal: ", end = " ")
78 for i in tree.postorder():
79     print(i.element(), end = " ")
80
```

```
84 tree = Tree()
85 print("Vertex: r a b c d e f g h i")
86 print("L.C    a c f g - i - - - ")
87 print("R.C    b d f h - - - - - ")
88 tree._add_root('r')
89 tree._add_left(tree.root(), e: 'a')
90 tree._add_right(tree.root(), e: 'b')
91 tree._add_left(tree.left(tree.root()), e: 'c')
92 tree._add_right(tree.left(tree.root()), e: 'd')
93 tree._add_left(tree.right(tree.root()), e: 'e')
94 tree._add_right(tree.right(tree.root()), e: 'f')
95 tree._add_left(tree.left(tree.left(tree.root()))), e: 'g')
96 tree._add_right(tree.left(tree.left(tree.root()))), e: 'h')
97 tree._add_left(tree.left(tree.right(tree.root()))), e: 'i')
98
99
100 #use the preorder traversal to print the tree
101 print("Preorder traversal: ", end = " ")
102 for i in tree.positions():
103     print(i.element(), end = " ")
104 print()
105
106 #use the post order traversal to print the tree
107 print("Postorder traversal: ", end = " ")
108 for i in tree.postorder():
109     print(i.element(), end = " ")
110 print()
111 print()
```

OUTPUT:

```
Vertex: r a b c d e f g h
L.C      a b - e - - - -
R.C      - c d f - g - h -
Preorder traversal:  r a b d c e g h f
Postorder traversal: d b h g e f c a r
```

```
Vertex: r a b c d e f g
L.C      a c - - - - -
R.C      b d e - - f g -
Preorder traversal:  r a c d b e f g
Postorder traversal: c d a g f e b r
```

```
Vertex: r a b c d e f
L.C      a - d f - - -
R.C      b c e - - - -
Preorder traversal:  r a c f b d e
Postorder traversal: f c a d e b r
```

```
Vertex: r a b c d e f g h i
L.C      a c f g - i - - -
R.C      b d f h - - - - -
Preorder traversal:  r a c g h d b e i f
Postorder traversal: g h c d a i e f b r
```